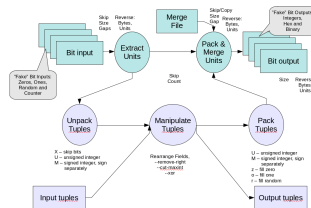


bdd — Bit Data Dump

High-performance CLI tool and Rust library to interpret, manipulate, stream, merge, and pack arbitrary-width bitstreams.

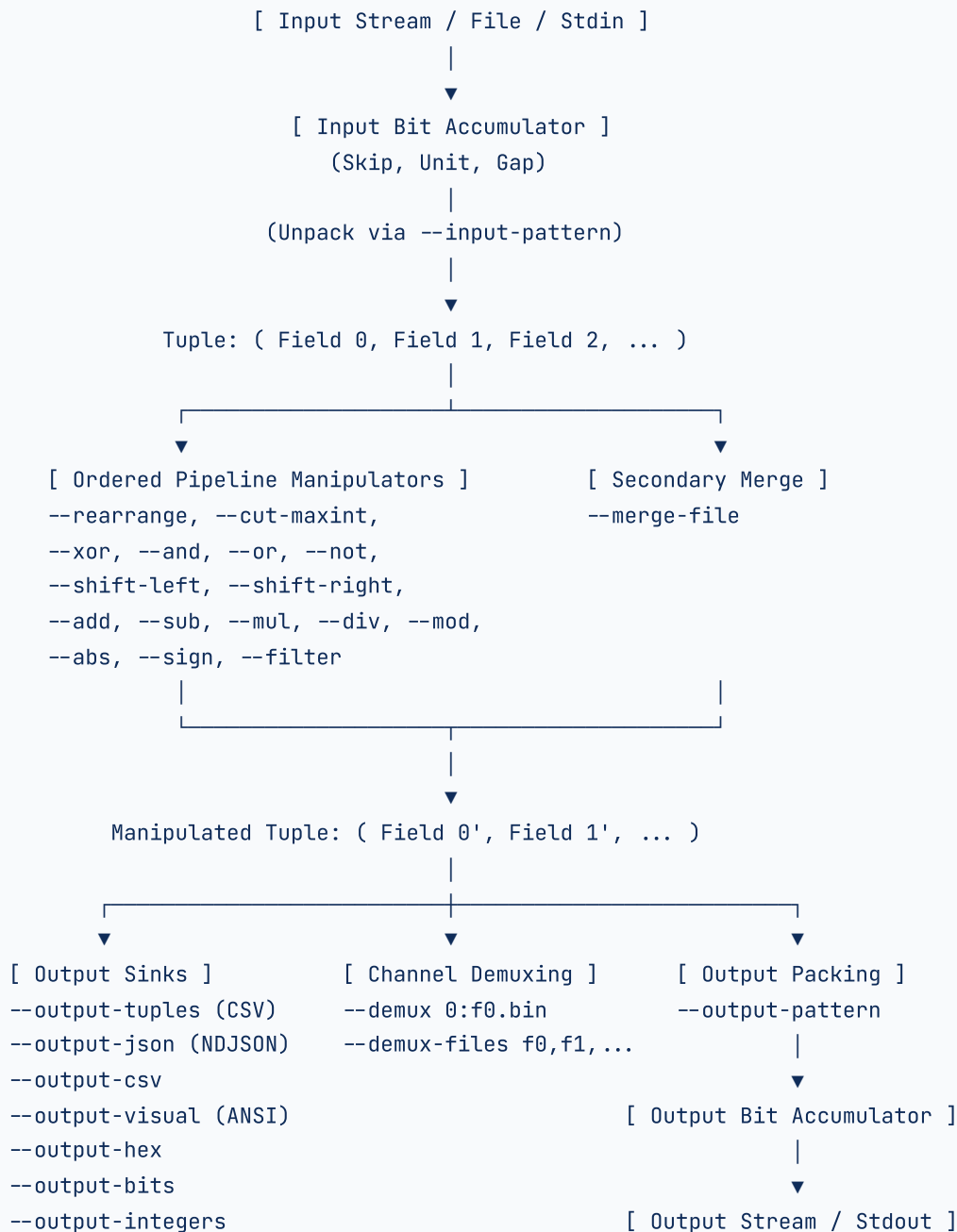
Originally created by Esa Turtiainen in Python 2 (2010), `bdd` was re-engineered in modern Rust for memory safety, 20-year maintainability, and multi-gigabit throughput with zero external C dependencies.



What is bdd?

Most Unix tools (`dd`, `hexdump`, `od`, standard shell pipes) operate strictly on byte (8-bit) or machine-word (16, 32, 64-bit) boundaries. Real-world binary streams—such as embedded sensor telemetry, cryptographic protocols, SDR/radio frames, network headers, compressed payloads, and legacy archives—frequently pack unaligned fields of arbitrary bit lengths (e.g., 1-bit flags, 3-bit opcodes, 12-bit ADC samples, or 57-bit mantissas).

`bdd` is a **binary-dd for bitstreams**: it allows you to slice, unpack, inspect, transform, interleave, and repack streams of any bit width with single-bit precision.



1. Anatomy of a Bit Stream

The Default Baseline: 8-Bit Streaming

The simplest invocation of `bdd` establishes its fundamental operating principle:

```
bdd < a > b
```

This command copies file `a` to file `b` in **default 8-bit units** (1 byte at a time) from standard input to standard output, without any modifications (acting like a byte-exact `cat` or `dd`).

By default:

- **Default Input Unit:** 8 bits (1 byte)
- **Default Output Unit:** 8 bits (1 byte)
- **Default Gaps & Skips:** 0 bits

Every slicing and packing feature in `bdd` builds on top of this baseline. For example, if you change `--input-unit=3` without setting an output unit:

```
# Extracts 3-bit values and expands each into an 8-bit byte with 5 leading zero bits:
bdd --input-unit=3 < a > b
```

Because the output unit remains at its default of 8 bits, each 3-bit input unit (`xxx`) is padded on the left with five zero bits (`00000xxx`) to produce an 8-bit byte on standard output.

Units, Skips, and Gaps

In `bdd`, all stream dimensions are measured strictly in **bits**, not bytes. A stream is modeled as a repeating series of **units**:

Stream Start



Skip Bits	Unit (Size)	Gap	Unit (Size)	Gap	...
-----------	-------------	-----	-------------	-----	-----

- `--input-unit=BITS` (`-u`): The number of bits in each repeating processing unit (default: `8`).
- `--input-skip-bits=BITS` : Initial offset in bits skipped before processing the first unit (default: `0`).
- `--input-skip-units=UNITS` : Skip initial units from stream start.
- `--input-gap=BITS` : Number of bits skipped *between* successive units (default: `0`).
- `--input-pregap=BITS` : Initial gap before the first unit. A convenient shorthand identical to setting `--input-skip-bits` and `--input-gap` together.
- `--input-assert-aligned` : Aborts with an error if the input stream terminates unaligned to a byte boundary.

Automatic Fast Seeking vs. Streaming Fallback

When extracting fields with `--input-skip-bits` or `--input-skip-units`, `bdd` automatically uses *O*(1) **filesystem seeking** by default whenever the input source is a seekable regular file or file descriptor (including `< file` shell redirection). Instead of transferring gigabytes from disk and discarding them byte-by-byte in memory, `bdd` jumps straight to the initial offset instantly.

If the input is non-seekable (such as a standard pipe `cat file | bdd`, FIFO, or socket), `bdd` seamlessly falls back to streaming sequential reads, discarding skipped bytes without failing or requiring separate flags.

To explicitly disable seeking and force sequential stream consumption across all inputs, pass `--no-seek` (or `--do-not-seek`, `--input-no-seek`, `--merge-no-seek`).

Slicing Bits from Byte Streams

Consider extracting specific sub-fields from a continuous byte stream:

Byte 0			Byte 1			
Pre-Gap	Unit	Gap	Unit	Gap	...	
(1 bit)	(2 bits)	(5 bits)	(2 bits)	(5 bits)		

```
bdd --input-pregap=1 --input-unit=2 --input-gap=5 --output-hex < input.bin
```

Common Slicing Recipes

- **Extract the Most Significant Bit of every byte** (expands 1 bit to an 8-bit byte `0x00` or `0x01`):

```
bdd --input-unit=1 --input-gap=7 --output-unit=8 < input.bin
```

- **Pack the Least Significant Bit of bytes into a dense bitstream** (e.g., 9 bytes with LSB=1 become two bytes `0xFF` and `0x80`):

```
bdd --input-pregap=7 --input-unit=1 --output-unit=1 < input.bin
```

Unit Sizing and Alignment Rules

- **Default Unit Size:** As established above, both input unit size and output unit size default to 8 bits (`bdd < a > b` copies bytes unchanged).
- **Expansion (Output > Input):** If the output unit is larger than the input unit, the value is treated as an integer and the left (most significant) bits are padded with zeros.
- **Truncation (Output < Input):** If the output unit is smaller than the input unit, the left (most significant) bits are truncated.
- **Stream Termination:** If the total bits written to an output byte stream are not a multiple of 8, the missing least-significant bits of the final byte are padded with zeros.

2. The Tuple Concept & Pattern Grammar

Rather than treating a bit unit as an opaque integer, `bdd` allows dividing a unit into named fields called a **Tuple** (analogous to a structured row in a relational database):

Raw Unit: 0 0 1 1 1 0 1 1 1 (9 bits)

Pattern: 3U 1x 2u 3M

(abs)]

| | | |

| | |

→ 3-bit Signed with Sign-Split → Yields TWO fields: [1 (sign), 1

| |

→ 2-bit Unsigned (Bit-Reversed) → Field value: 1

|

→ 1-bit Skip → Discarded (not in tuple)

|

→ 3-bit Unsigned Integer → Field value: 1

Unpacked Tuple: (1, 1, 1, 1)

Pattern Specifiers Reference

Every field in an `--input-pattern` or `--output-pattern` is specified as `<bits><type>` (with optional repetition multipliers like `4*8B` or `2*(4U4u)`):

CODE	NAME	DESCRIPTION	ALLOWED BIT WIDTHS	VALID CONTEXT
nU / nB	Unsigned Integer / Byte	Standard big-endian unsigned integer	Any positive integer	Input / Output
nu / nb	Unsigned (Reversed)	Unsigned integer with reversed bit order	Any positive integer	Input / Output
nS	Signed Integer	Two's complement signed integer (returns negative values)	Any positive integer	Input / Output
ns	Signed (Reversed)	Two's complement with reversed bit order	Any positive integer	Input / Output
nM	Sign-Split Integer	Two's complement integer split into two fields: [sign, abs(val)]	Any positive integer	Input / Output
nm	Sign-Split (Reversed)	Sign-split integer with reversed bit order	Any positive integer	Input / Output
32F / 32f	32-bit Float	IEEE 754 single-precision float (f reverses bits)	Exactly 32 bits	Input / Output
64D / 64d	64-bit Double	IEEE 754 double-precision float (d reverses bits)	Exactly 64 bits	Input / Output
16H / 16h	Half-Precision Float	IEEE 754 half-precision float (FP16)	Exactly 16 bits	Input / Output
16Y / 16y	Bfloat16	Google Brain Bfloat16 float (BF16)	Exactly 16 bits	Input / Output
8E / 8e	OCP FP8 (E4M3)	Open Compute / NVIDIA Hopper FP8 E4M3FN	Exactly 8 bits	Input / Output
8Q / 8q	OCP FP8 (E5M2)	Open Compute / NVIDIA Ada FP8 E5M2	Exactly 8 bits	Input / Output
6E / 6e	OCP FP6 (E3M2)	Open Compute Microscaling FP6 E3M2	Exactly 6 bits	Input / Output
4E / 4e	OCP / NVFP4 (E2M1)	NVIDIA Blackwell / OCP Microscaling FP4 E2M1	Exactly 4 bits	Input / Output
nC / nc	Characters / Bytes	Raw byte/character sequence (retains byte fidelity)	Multiples of 8 bits	Input / Output

CODE	NAME	DESCRIPTION	ALLOWED BIT WIDTHS	VALID CONTEXT
<code>nx</code>	Skip / Discard	Discards n bits from input without placing them in the tuple	Any positive integer	Input only
<code>nz</code>	Fill Zeros	Inserts n constant zero bits	Any positive integer	Output only
<code>no</code>	Fill Ones	Inserts n constant one bits	Any positive integer	Output only
<code>nr</code>	Fill Random	Inserts n pseudo-random bits	Any positive integer	Output only

Pattern Repetition Multipliers

Patterns support multiplier syntax for repetitive fields and tensor arrays:

- `4*8B` : Expands to `8B8B8B8B` (4 bytes).
- `10*16H` : Expands to ten 16-bit half-precision floats.
- `2*(4U4u)` : Expands to `4U4u4U4u`.

Distinction Between `s` and `M`

- **`s` (Signed Integer)**: Directly represents negative values (e.g., `-5`). When printed via `--output-tuples`, it outputs `-5`.
- **`M` (Magnitude & Sign Split)**: Decodes a signed two's complement number into **two separate unsigned fields**:
 1. `sign` : `0` for positive, `1` for negative.
 2. `magnitude` : `abs(value)`. This allows arithmetic, filtering, or routing based on sign and absolute value independently.

3. Flexible Pipeline & Tuple Manipulation

Once unpacked into a tuple, fields can be transformed using pipeline manipulators executed in the exact order specified on the command line. Fields are referenced by index starting from `0` (or negative index from end):

Pipeline Manipulators

- `--rearrange=F0,F1,...` : Reorders, duplicates, or drops fields (e.g., `--rearrange=1,0`).
- `--cut-maxint=FIELD,MAX[,MODE]` : Bounds or rounds field value within `[-MAX, MAX]` (or `[0, MAX]` if unsigned). `MODE` can be specified as `,MODE` or `:MODE`. Supported Rust rounding and overflow

modes:

- `saturate` / `clamp` (default): Clamps out-of-bounds values to `MAX` or `-MAX`.
 - `wrap` / `wrapping`: Wraps values around using modular arithmetic (`[0, MAX]` for unsigned, `[-MAX, MAX]` for signed).
 - `zero` / `reset`: Sets out-of-bounds values to 0.
 - `drop` / `filter` / `checked`: Discards the tuple entirely if the value exceeds bounds.
 - `trunc` / `truncate`: Truncates magnitude toward zero.
 - `floor`: Rounds toward negative infinity.
 - `ceil`: Rounds toward positive infinity.
 - `round`: Rounds to nearest neighbor, ties away from zero.
 - `round_ties_even` / `even` / `bankers`: Rounds to nearest neighbor, ties to nearest even digit (Rust `f64::round_ties_even`).
-
- `--shift-left=FIELD,BITS`: Bitwise left-shifts field by `BITS`.
 - `--xor=FIELD,PARAM`: Bitwise XOR with parameter (supports hex `0x...`, bin `0b...`, or bit-count mask).
 - `--and=FIELD,PARAM`: Bitwise AND with parameter.
 - `--or=FIELD,PARAM`: Bitwise OR with parameter.
 - `--not=FIELD`: Bitwise NOT (inverts field bits).
 - `--add=FIELD,PARAM`: Adds integer parameter to field.
 - `--sub=FIELD,PARAM`: Subtracts integer parameter from field.
 - `--mul=FIELD,PARAM`: Multiplies field by parameter.
 - `--div=FIELD,PARAM`: Integer division of field by parameter.
 - `--mod=FIELD,PARAM`: Modulo of field by parameter.
 - `--abs=FIELD`: Converts a signed field to its absolute value (syntax: `--abs 0` or legacy `--abs 0,0`).
 - `--sign=FIELD`: Isolates sign bit (`0` = positive, `1` = negative).
 - `--filter=FIELD,OP,VALUE`: Drops tuples where predicate is false (`=`, `≠`, `<`, `≤`, `>`, `≥`).

Ordered Pipeline Example

Chain multiple transformations in arbitrary sequence:

```
# Add 10, multiply by 2, and filter values greater than 25:
bdd --input-counter --count 10 --input-pattern 8U \
    --add 0,10 --mul 0,2 --filter "0,>,25" --output-json
```


Advanced Bit-Slicing Recipe

Problem from presentation: Input is a stream of signed bytes (`8M`). We want to clamp the value so -128 becomes -127 , isolate the magnitude, and extract only the most significant bit of the magnitude:

```
bdd --input-pattern=8M --rearrange=-1 --cut-maxint=0,127 --remove-right=0,6 --output-unit=1 <
input.bin
```

Explanation:

1. `8M` produces `[sign, magnitude]`.
2. `--rearrange=-1` selects only the last field (magnitude).
3. `--cut-maxint=0,127` clamps the magnitude to 127.
4. `--remove-right=0,6` shifts right by 6 bits, leaving the 7th bit (MSB of magnitude).
5. `--output-unit=1` outputs the single bit.

4. Synthetic ("Fake") Streams & Text Formats

`bdd` can synthesize bitstreams without requiring input files, and output in human-readable or script-friendly formats:

Synthetic Stream Sources

- `--input-zeros ( -0 )`: Endless stream of zero bits.
- `--input-ones ( -1 )`: Endless stream of one bits.
- `--input-random ( -r )`: Pseudo-random bits (PRNG).
- `--input-counter ( -c )`: Sequential integer counter starting at 0, incrementing by 1 per unit.
- `--count=N`: Limit processing to N units (0 = infinite).
- `--skip=N`: Skip N initial units before processing.

Human-Readable & Script Sinks

- `--output-hex ( -x )`: Formatted hexadecimal representation per unit, grouped neatly into power-of-two line widths.
- `--output-bits ( -b / --output-bit )`: ASCII bit strings (`'0'` and `'1'`) per unit.
- `--output-integers ( -i / --output-integer )`: One unsigned decimal integer per line (ideal for Unix pipelines: `awk`, `sort`, `uniq`).
- `--output-tuples ( -T / --output-tuple )`: Comma-separated values per unit.
- `--output-json`: Newline-delimited JSON (NDJSON) record per tuple (ideal for `jq`, Python, databases).
- `--output-csv`: Comma-separated values (with optional `--csv-header="col1,col2"`).
- `--output-visual`: Interactive colorized terminal dump rendering unaligned field slices in alternating ANSI colors.

- `--input-tuples (-t)`: Ingest comma-separated values directly from stdin/file into tuples.

Packing from Text & Tuples

```
# Pack octal numbers from text tuples into binary:
printf "1,2,3\n3,7,7\n" | bdd --input-tuples --output-pattern='2U3U3U' | od -t o1
# Outputs: 123 377

# Pack two 4-bit numbers into hex bytes:
echo -en "1,2\n3,4" | bdd --input-tuples --output-pattern=4U4U | od -t x1
# Outputs: 12 34

# Pack text numbers into 64-bit IEEE double-precision floats:
printf "1.0\n2.0\n" | bdd --input-tuples --output-pattern=64D > doubles.bin
```

5. De-mystifying Bit Reversals

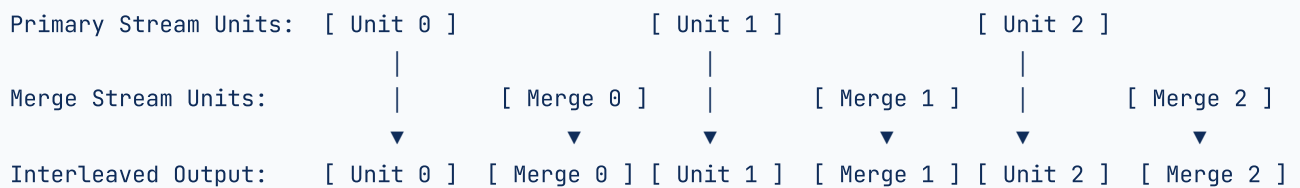
Reversing bits in binary processing can easily become confusing. `bdd` cleanly isolates bit reversals into four distinct operational layers:

```
[ Input Byte Stream ]
    |
(1)  |→ --reverse-input-bytes   (Reverses bits within each 8-bit byte read)
    ▼
[ Input Units ]
    |
(2)  |→ --reverse-input-units   (Reverses all bits across the entire unit)
    ▼
[ Tuple Fields ]
    |
(3)  |→ Lowercase Pattern Codes (u, s, m, c - reverses bits within specific field)
    ▼
[ Output Units ]
    |
(4)  |→ --reverse-output-units (Reverses all bits across the entire output unit)
    ▼
[ Output Byte Stream ]
    |
(5)  |→ --reverse-output-bytes (Reverses bits within each 8-bit byte written)
```

- `--input-little-endian`: Shorthand combining `--reverse-input-bytes` and `--reverse-input-units`.
- `--output-little-endian`: Shorthand combining `--reverse-output-bytes` and `--reverse-output-units`.

6. Stream Merging & Interleaving

The merge stream reads a secondary file and interleaves its data into the primary stream:



- `--merge-file=PATH` : Interleave data from a secondary file (`"-"` for stdin).
- `--merge-unit=BITS` : Size of merge units in bits (default: 8).
- `--merge-copy-first=BITS` : Copies an initial bit header/preamble from the merge file *before* starting the interleaved loop.

Example: Hex Dump with Interleaved Memory Addresses

Interleave a 12-bit linear address counter with actual data bytes from `/etc/passwd` :

```
bdd --input-counter --count=16 --input-unit=12 --output-unit=12 --merge-file=/etc/passwd --
output-hex
```

Output:

```
000 72 001 6f 002 6f 003 74 004 3a 005 78 006 3a 007 30
008 3a 009 30 00a 3a 00b 72 00c 6f 00d 6f 00e 74 00f 3a
```

7. Command Line Options Reference

Usage: bdd [OPTIONS] [FILE]

Arguments:

[FILE] Input file (defaults to standard input '-')

Input Unit & Pattern Options:

-p, --input-pattern <PATTERN>	Bit pattern to unpack input (e.g. "3U1x2u3M")
-u, --input-unit <BITS>	Input unit size in bits (shorthand for <BITS>U)
--input-skip-bits <BITS>	Initial bit offset before first unit [default: 0]
--input-skip-units <UNITS>	Skip initial N units from input stream [default: 0]
--input-gap <BITS>	Bit gap skipped between units [default: 0]
--input-pregap <BITS>	Shorthand for initial skip & gap [default: 0]
--input-assert-aligned	Error if EOF is not byte-aligned
--no-seek, --do-not-seek	Globally disable seeking on all inputs (force streaming read)
--input-no-seek	Disable seeking specifically on primary input
--input-use-seek	Explicitly enable seeking on input (default: true)

Synthetic Stream Sources:

-c, --input-counter	Generate sequential counter numbers (0, 1, 2...)
-0, --input-zeros	Generate endless stream of zero bits
-1, --input-ones	Generate endless stream of one bits
-r, --input-random	Generate pseudo-random bits
-t, --input-tuples	Read comma-separated tuple lines from text input
--skip <UNITS>	Skip initial N units [default: 0]
--count <COUNT>	Process at most N units (0 = infinite) [default: 0]

Bit Reversal Options:

--reverse-input-bytes	Reverse bit order within each input byte
--reverse-input-units	Reverse bit order across entire input unit
--input-little-endian	Combined byte and unit reversal for input
--reverse-output-bytes	Reverse bit order within each output byte
--reverse-output-units	Reverse bit order across entire output unit
--output-little-endian	Combined byte and unit reversal for output

Tuple Manipulators:

--rearrange <FIELDS>	Reorder output fields (e.g. "1,0" or "-1,0")
--cut-maxint <F,MAX[,MODE]>	Clamp or round field F to [-MAX, MAX] (modes: saturate, wrap, zero, drop, trunc, floor, ceil, round, round_ties_even)
--remove-right <F,BITS>	Right-shift field F by BITS
--shift-right <F,BITS>	Right-shift field F by BITS (synonym)
--shift-left <F,BITS>	Left-shift field F by BITS
--xor <F,PARAM>	Bitwise XOR field F with PARAM (or mask of N ones)
--and <F,PARAM>	Bitwise AND field F with PARAM
--or <F,PARAM>	Bitwise OR field F with PARAM
--not <FIELD>	Bitwise NOT (inverts bits of field F)
--add <F,PARAM>	Add PARAM to field F
--sub <F,PARAM>	Subtract PARAM from field F

<code>--mul <F,PARAM></code>	Multiply field F by PARAM
<code>--div <F,PARAM></code>	Integer division of field F by PARAM
<code>--mod <F,PARAM></code>	Modulo of field F by PARAM
<code>--abs <FIELD></code>	Replace signed field F with abs(F)
<code>--sign <FIELD></code>	Replace signed field F with sign bit (0/1)
<code>--filter <F,OP,VAL></code>	Filter tuples where predicate is false (<code>=</code> , <code>≠</code> , <code><</code> , <code>≤</code> , <code>></code> , <code>≥</code>)

Output Unit & Pattern Options:

<code>--output-pattern <PATTERN></code>	Bit pattern to pack output (supports AI floats & multipliers)
<code>--output-unit <BITS></code>	Output unit size in bits (shorthand for <code><BITS>U</code>)
<code>-x, --output-hex</code>	Output units as formatted hexadecimal strings
<code>-b, --output-bits</code>	Output units as ASCII bit strings ('0' and '1')
<code>-i, --output-integers</code>	Output unsigned integer per unit (one per line)
<code>-T, --output-tuples</code>	Output fields as comma-separated tuples
<code>--output-json</code>	Output tuples as newline-delimited JSON (NDJSON)
<code>--output-csv</code>	Output tuples as CSV
<code>--csv-header <HEADER></code>	Optional CSV column header row
<code>--output-visual</code>	Colorized ANSI terminal dump of unaligned fields

Demuxing & Channel Splitting:

<code>--demux <FIELD:PATH></code>	Route individual tuple field to a dedicated output file
<code>--demux-files <PATHS></code>	Comma-separated list of output files for fields 0, 1, 2...
<code>--input-repeat <COUNT></code>	Repeat input stream N times (0 = infinite)

Merge Options:

<code>--merge-file <PATH></code>	Interleave stream from secondary file
<code>--merge-unit <BITS></code>	Bit width of each merge unit [default: 8]
<code>--merge-copy-first <BITS></code>	Copy initial header bits from merge file first
<code>--merge-no-seek</code>	Disable seeking specifically on merge file
<code>--merge-use-seek</code>	Explicitly enable seeking on merge file (default: true)

General:

<code>-h, --help</code>	Print help
<code>-V, --version</code>	Print version

8. Channel Demuxing & Splitting

When processing multiplexed packet formats or interleaved bitstreams (e.g. audio + video, header + payload), `bdd` can demux fields into independent files or pipes:

```
# Split interleaved 16-bit audio and 32-bit video into separate streams:
bdd --input-pattern=16B32B \
    --demux 0:audio.raw \
    --demux 1:video.raw < input.bin

# Alternatively, using --demux-files shorthand:
bdd --input-pattern=16B32B --demux-files=audio.raw,video.raw < input.bin
```

9. Programmatic Interfaces (C Header & Python)

`bdd` exports clean C-ABI symbols in `libbdd.so` and includes an official C header ([include/bdd.h](#)) and a zero-dependency Python wrapper ([python/bdd.py](#)):

Python (`ctypes`)

```
from bdd import Bdd

b = Bdd()

# Fast hardware bit-reversal:
assert b.reverse_bits(0x0F, 8) == 0xF0

# Unpack bitfields into Python tuple:
fields = b.unpack("4U4U", 0xA5)    # → (10, 5)

# Pack Python tuple into raw integer:
unit = b.pack("4U4U", (10, 5))     # → 0xA5

# Native AI float conversion (FP16, BF16, FP8, FP4):
val = b.decode_f16(0x3C00)         # → 1.0
bits = b.encode_f16(1.0)           # → 0x3C00
```

C / C++ API

```
#include "bdd.h"
#include <stdio.h>

int main() {
    uint64_t fields[2];
    bdd_unpack_u64("4U4U", 0xA5, fields, 2);
    printf("Field 0: %lu, Field 1: %lu\n", fields[0], fields[1]); // 10, 5
    return 0;
}
```

10. UTF-8 Stream Processing & Architectural Study

For an in-depth analysis of UTF-8 bitstream hazards, byte-alignment constraints, continuation header preservation, and Unicode scalar value processing in `bdd` , consult the technical report:

- `docs/utf8_study.md`

11. Performance & Throughput Benchmarks

`bdd` achieves high throughput across arbitrary bit boundaries, balancing hardware register acceleration for sub-64-bit units with arbitrary-precision arithmetic for large bignum fields.

Measured via `make bench` (`benches/throughput.rs`) on Linux x86_64:

OPERATION	TOTAL VOLUME	THROUGHPUT (BITS/S)	THROUGHPUT (BYTES/S)	NOTES
Hardware 64-bit Bit Reversal	128 Mbits	16.00 Gbps	2,000 MB/s	Direct CPU <code>u64::reverse_bits()</code>
Bignum 1024-bit Packing/Unpacking	10.2 Mbits	5.27 Gbps	658.6 MB/s	Large-block bignum bitfield packing
Bignum 256-bit Packing/Unpacking	12.8 Mbits	1.90 Gbps	237.2 MB/s	SHA-256 size field packing/unpacking
Synthetic 8-bit Linear Stream	16.0 Mbits	234.5 Mbps	29.3 MB/s	Continuous bit generation & sink flush
1-bit Single-Bit Resolution Stream	0.5 Mbits	43.3 Mbps	5.4 MB/s	Single-bit slice accumulation & packing
Bignum 1024-bit Bit Reversal	5.1 Mbits	39.4 Mbps	4.9 MB/s	Full arbitrary-precision bit reversal
Unaligned 3-bit to 8-bit Extraction	3.0 Mbits	39.0 Mbps	4.9 MB/s	Cross-byte boundary accumulation
Tuple Pipeline (<code>2U3U3U</code> → Rearrange)	2.0 Mbits	29.7 Mbps	3.7 MB/s	Multi-field unpack, reorder & repack

12. Architecture & Codebase Design

The Rust implementation is organized cleanly into modular crates:

```
src/
├─ lib.rs          # Public library crate interface
├─ main.rs         # Thin 18-line executable wrapper
├─ error.rs        # Strongly-typed BddError hierarchy
├─ counter.rs      # Unit and skip counting logic
├─ field.rs        # Arbitrary-precision Field enum & hardware bit-reversals
├─ float_types.rs  # AI/GPU float codecs (FP16, BF16, FP8 E4M3/E5M2, FP6, FP4)
├─ ffi.rs          # C-ABI export symbols for native integration
├─ pattern.rs      # Grammar parser, multipliers, TupleUnpacker & TuplePacker
├─ stream.rs       # Stream generators (File, Counter, Zeros, Ones, Random, Tuples)
├─ sink.rs         # Output writers (Binary, Hex, Bit, Integer, CSV, NDJSON, ANSI Visual)
├─ manipulator.rs  # Ordered pipeline transformations (Arithmetic, Bitwise, Filter)
├─ cli.rs          # Clap CLI definition & validation rules
├─ engine.rs       # End-to-end pipeline execution orchestrator
include/
├─ bdd.h           # C/C++ API header
python/
├─ bdd.py          # Zero-dependency Python ctypes wrapper
```

Key Design Principles

1. **Strict Type Safety:** All errors flow through `BddError`. Functions return `Result<T, BddError>` instead of panicking or calling `std::process::exit`.
2. **Fast-Path Bit Reversal:** Sub-64-bit integer bit reversals execute via direct hardware `u64::reverse_bits()`, falling back to `BigUint` bit arithmetic only when necessary.
3. **Byte-Level String Integrity:** The `C` and `c` pattern types store raw bytes internally (`Field::Bytes`) rather than lossy UTF-8 conversions, guaranteeing bit-perfect roundtrips.
4. **Automated Verification:** Integrated test runner runs native Rust unit tests, bignum tests, and legacy golden-file integration tests.

13. Development, Testing & Documentation

Run the test suite:

```
make test
# or: cargo test --all-targets --all-features
```

Run benchmarks:


```
make bench
# or: cargo bench
```

Run code linter and formatting checks:

```
make lint
make check-fmt
```

Documentation & PDF Generation

All project documentation compiles into clean, print-ready vector PDF and HTML files:

- [README.pdf](#) : Generated from `README.md` via headless Chromium / Puppeteer with vector math, diagrams, and GFM styling.
- [docs/bdd.1.pdf](#) : Unix manual page rendered as clean vector PDF via `groff` and `ps2pdf` .
- [docs/bdd.1.html](#) : Unix manual page rendered as standalone HTML.
- [docs/Presentation.pdf](#) : Architectural slide deck compiled from OpenDocument Presentation (`legacy/old_src/Presentation.odp`) via headless LibreOffice.

To regenerate all documentation artifacts in one command:

```
make docs
```

License

GPL-3.0-or-later. Original author: Esa Turtiainen.